Multimodal Trajectory Prediction with a GRU Decoder

Ashok Ramasamy
Dept. of Electrical Engineering
Chalmers University of Technology
Gothenburg, Sweden
ashokr@chalmers.se

Danilo Rameschandra
Dept. of Mechanical Engineering
Chalmers University of Technology
Gothenburg, Sweden
danilora@chalmers.se

***Abstract*—This project investigates multimodal vehicle trajectory prediction on the nuScenes dataset. Starting from a target-only MLP, we add graph-based agent interactions, multiple output modes, road map context and a GRU decoder one at a time to measure what each contributes. Map context gives the clearest gain, reducing minFDE_6 by 13.4%, and the recurrent decoder further reduces it by 6.1%, with the largest gains on turning trajectories. However, the final model barely uses neighboring agents, likely because its edge features ignore relative motion, and sharp turns remain difficult.**

***Index Terms*—Multimodal Trajectory Prediction, Map Context, Graph Modeling, Autonomous Driving**

I. Introduction

Vehicle trajectory prediction is an important task in autonomous driving, where the future motion of surrounding road users must be estimated for safe planning. Because future behavior is inherently uncertain, recent methods often predict multiple plausible trajectories rather than a single outcome [1]. Other approaches explicitly model interactions between nearby agents [2] or incorporate structured lane and map information to constrain possible motion [3].

The aim of this project is to study how these components affect the prediction of a designated target vehicle. We use the nuScenes prediction dataset [4], which provides annotated urban scenes with agent trajectories and map information. Starting from a target-only predictor, we progressively introduce graph-based interactions, multimodal outputs, lane context, and a recurrent GRU decoder, and evaluate their effect using displacement-based metrics.

II. Related Work

Early trajectory predictors used recurrent networks to extend an agent's past motion into the future [5]. However, a single predicted path tends to average out distinct behaviors, for example ending up halfway between turning and going straight [6]. To address this, MTP [6] predicts several trajectories with probabilities, while CoverNet [1] instead selects the most likely paths from a predefined set of trajectories the vehicle can physically drive. Since drivers also react to one another, AgentFormer [2] and Trajectron++ [7] model interactions between agents, the latter using a graph with a recurrent decoder. Both reported gains on nuScenes.

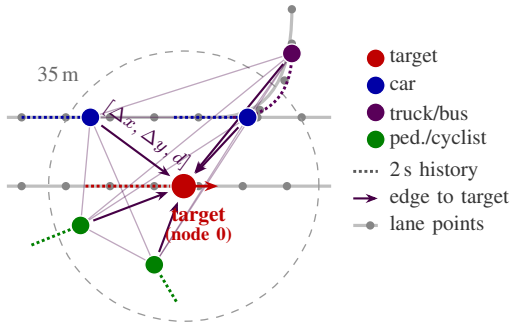


Fig. 1. Example scene in the target-centric frame, with the interaction graph around the target (node 0) and the sampled lane points.

Vehicles also largely follow lanes, so road geometry strongly constrains plausible motion. LaneGCN [3] encodes the HD map as a lane graph that exchanges information with agents, while VectorNet [8] represents both map and trajectories as vectors, matching image-based encodings at lower computational cost.

Most of these models combine several ideas at once, so it is unclear how much multiple trajectories, agent interactions and map context each contribute on their own. We therefore add them one at a time to a simple model that only uses the target's past motion, and measure the effect of each.

III. Method

A. Dataset and Scene Representation

We use the nuScenes prediction dataset [4]. Each sample contains 2 s of past motion (5 positions) for a target vehicle, and the task is to predict its next 6 s (12 positions) at 2 Hz. As is common in prior work [8], each scene is centered on the target and rotated so that it faces along the x-axis (Fig. 1), which allows the model to learn how vehicles move relative to their own heading, rather than where they are on the map.

Around the target, we build an interaction graph from all agents within 35 m, a radius chosen by comparing several values on the validation set. Only agents with a complete history are kept, and every pair is connected by directed edges that store their offset and distance $\mathbf{e}_{ij} = [\Delta x, \Delta y, d]$. Road layout is added as points sampled along lane and lane-connector centerlines within 50 m. After preprocessing, the

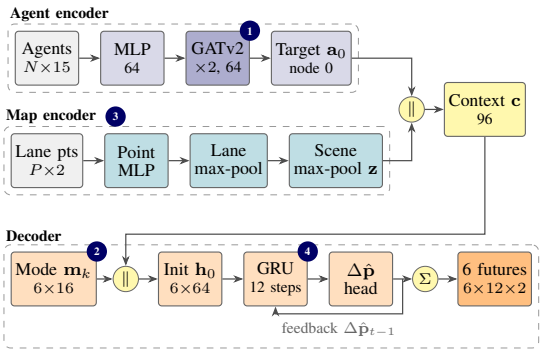


Fig. 2. Model architecture. Numbers mark the components added incrementally to the target-only baseline: (1) GATv2 interaction layers, (2) multimodal output with six modes, (3) map encoder, and (4) GRU decoder, which replaces the one-shot MLP decoder.

TABLE I
INTERACTION MODELING (TEST SET).

Model	ADE [m]	FDE [m]
Target-only MLP	3.824 ± 0.011	9.046 ± 0.002
GATv2	**3.779 ± 0.043**	**8.927 ± 0.078**

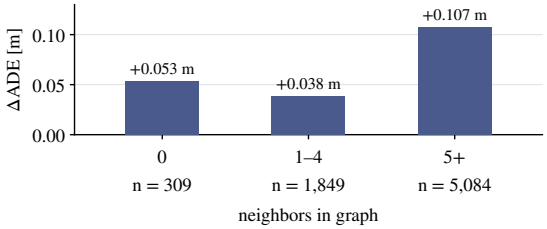


Fig. 3. ADE reduction of GATv2 over the target-only MLP by number of neighbors (one seed).

training, validation and test sets contain 25,711, 6,685 and 7,242 samples, taken from the official nuScenes `train`, `train_val` and `val` splits, respectively.

B. Model Architecture

The model (Fig. 2) consists of an agent encoder, a map encoder and a decoder. Each agent i is described by a 15-D input $\mathbf{x}_i$, its five flattened past positions together with a one-hot encoding of its category, which a linear layer with ReLU maps to a 64-D feature vector. Two single-head GATv2 layers [9], each followed by ReLU, then update these features with information from neighboring agents. Each update is a weighted average over the neighbors, with weights learned from both agents and their edge features $\mathbf{e}_{ij}$, so that more relevant neighbors contribute more. Unlike the original GAT [10], where all agents rank their neighbors in the same way, GATv2 lets each agent rank them differently, which matters in traffic since the same vehicle can be relevant to one agent and irrelevant to another.

Inspired by VectorNet [8], the map encoder passes every lane point through a shared MLP and max-pools the results over the points of each lane, giving one vector per lane, and again over all lanes, giving a 32-D map embedding $\mathbf{z}$. This pooling makes $\mathbf{z}$ independent of how many points and lanes a scene contains. The target's features after the GATv2 layers, $\mathbf{a}_0$, are concatenated with $\mathbf{z}$ into the 96-D scene context $\mathbf{c} = [\mathbf{a}_0 \,\|\, \mathbf{z}]$, while models without the map use $\mathbf{c} = \mathbf{a}_0$.

The decoder outputs $K = 6$ trajectories, or modes ($K = 1$ for the deterministic variants). The one-shot decoder is an MLP with one 128-D hidden layer that predicts all $K\times12\times2$ future coordinates at once from $\mathbf{c}$. The GRU decoder [11] instead predicts one displacement per step and is run once per mode. Its initial hidden state $\mathbf{h}_0^k = \tanh(\mathbf{W}[\mathbf{c} \,\|\, \mathbf{m}_k] + \mathbf{b})$ combines the context with a learned 16-D embedding $\mathbf{m}_k$ for each mode, so that the six runs start differently and produce different trajectories. At each step, the GRU receives the previously predicted displacement, starting from $\Delta\hat{\mathbf{p}}_0^k = \mathbf{0}$, and the positions are the sum of all displacements so far,

$$\mathbf{h}_t^k = \mathrm{GRU}\big(\Delta\hat{\mathbf{p}}_{t-1}^k, \mathbf{h}_{t-1}^k\big), \qquad \hat{\mathbf{p}}_t^k = \textstyle\sum_{\tau=1}^{t} \Delta\hat{\mathbf{p}}_\tau^k, \tag{1}$$

where $\Delta\hat{\mathbf{p}}_t^k$ is obtained from $\mathbf{h}_t^k$ by a linear head and $t = 1, \ldots, 12$.

C. Training and Evaluation

We use a winner-takes-all loss [6], where only the trajectory closest to the ground truth $\mathbf{p}$ contributes,

$$\mathcal{L} = \min_k \frac{1}{T} \sum_{t=1}^{T} \left\| \hat{\mathbf{p}}_t^k - \mathbf{p}_t \right\|_2, \quad T = 12. \tag{2}$$

All models are trained for 30 epochs with Adam, using a learning rate of 10^{-3} and a batch size of 64, except for the decoder comparison in Section IV-D, which uses 60 epochs. Each model is trained with three seeds, and the best validation checkpoint of each is evaluated on the test set, with all tables reporting the mean ± standard deviation over the three seeds. Performance is measured with minADE_K, as in (2), and minFDE_K, the error of the closest trajectory at the final step T. For $K = 1$, these metrics reduce to the standard ADE and FDE.

IV. Results and Discussion

A. Do Agent Interactions Help?

We first ask whether surrounding agents improve prediction beyond the target's own history. Adding GATv2 to the target-only MLP reduces ADE by 1.2% and FDE by 1.3% (Table I).

This average gain, however, varies with scene density. With $\Delta\mathrm{ADE} = \mathrm{ADE}_{\mathrm{MLP}} - \mathrm{ADE}_{\mathrm{GAT}}$, the largest gain occurs in scenes with five or more neighbors (Fig. 3). Yet even samples without neighbors improve slightly, suggesting that part of the GATv2 gain cannot be attributed solely to interaction information.

To test whether the final model relies on neighbors at all, we removed all neighbor nodes at inference from the map-aware GRU model while keeping the target and map unchanged. For all three seeds, minADE_6 changed by less than 0.001 m. Although interaction modeling gives a small

TABLE II
MAP CONTEXT (TEST SET).

Model	minADE_6 [m]	minFDE_6 [m]
No map	1.714 ± 0.007	3.653 ± 0.028
Map	**1.555 ± 0.014**	**3.162 ± 0.035**

TABLE III
DECODER COMPARISON (TEST SET, 60 EPOCHS).

Decoder	minADE_6 [m]	minFDE_6 [m]
One-shot MLP	1.533 ± 0.003	3.121 ± 0.012
GRU	**1.474 ± 0.016**	**2.931 ± 0.044**

gain, the final model is thus almost insensitive to neighbor information, which led us to examine road-layout information next.

B. Does Map Context Help?

Before adding the map, we extended the GATv2 model from one to six predicted trajectories. This alone lowers the error substantially, since the best of six modes can match the observed maneuver even when the future is ambiguous. As vehicle motion is largely constrained by the surrounding road geometry, we then tested whether lane information improves this six-mode model further. Adding the map encoder reduces minADE_6 by 9.3% and minFDE_6 by 13.4% (Table II). The larger reduction in minFDE_6 suggests that lane context is useful for constraining where the target can end its trajectory over the 6 s horizon. This gain is substantially larger than the gain of interaction modeling, which is consistent with the importance of map information in LaneGCN [3] and VectorNet [8]. These averages, however, do not show where the remaining errors occur, which motivated a closer look at the cases in which the map-aware model fails.

C. Where Does the Model Fail?

To locate the remaining errors, we grouped the test samples by the endpoint angle $\theta = |\mathrm{atan2}(y_T, x_T)|$ of the ground-truth final position, which measures how much the target's direction changes over the 6 s horizon. The error grows strongly with this angle (Fig. 4). For the map-aware model, minFDE_6 rises from 2.56 m for near-straight motion (0–15°) to 7.30 m for 30–60° and 9.04 m above 60°, making turns the main failure mode.

We first checked whether the model was simply undertrained, but training for 60 instead of 30 epochs gave little overall gain. Oversampling turning samples during training then improved the 30–60° bin, but worsened the overall error and did not help the sharpest turns. Since neither longer training nor reweighting toward turns solved the problem, we suspected that the limitation lies in how the trajectory is decoded, as the one-shot MLP predicts all 12 positions at once without any notion of their order in time.

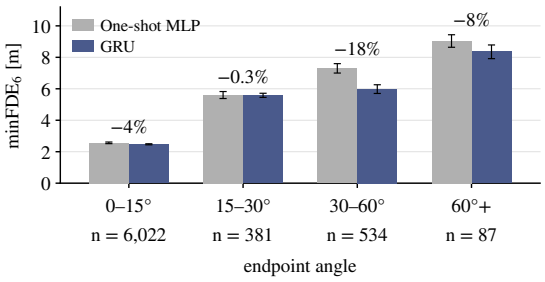


Fig. 4. Test minFDE_6 by endpoint angle for the one-shot and GRU decoders. Samples moving less than 5 m are excluded. Error bars show the standard deviation over three seeds.

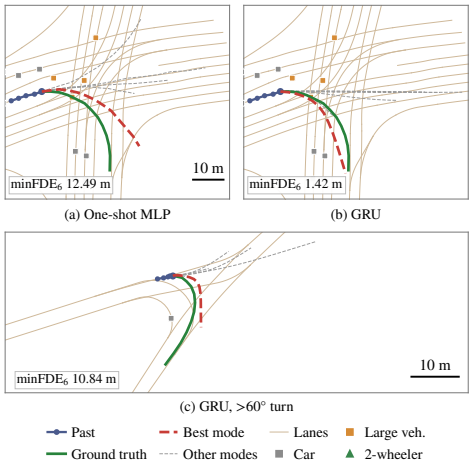


Fig. 5. Qualitative examples, chosen for clarity. (a, b) The same turn with both decoders. (c) A sharper turn where the GRU fails.

D. Does Recurrent Decoding Help?

We therefore replaced the one-shot decoder with the GRU decoder, training both for 60 epochs (Table III). The GRU reduces minADE_6 by 3.8% and minFDE_6 by 6.1%, and the per-bin results in Fig. 4 show that the largest relative gains occur on turning trajectories, with an 18% lower minFDE_6 for 30–60°. Predicting the trajectory step by step therefore appears to suit turns, where the heading changes gradually over the horizon. Above 60°, however, the improvement drops to 8% and the error remains high, so sharp turns are improved but not solved. Fig. 5 illustrates both cases.

V. CONCLUSIONS

Starting from a simple MLP that only uses the target's own past motion, we added interaction modeling, multiple output modes, map context and a GRU decoder one at a time. The results indicate that road geometry provides the clearest benefit, while the final model barely uses neighboring-agent information. One possible explanation is that the current edge features encode only relative position and distance, without capturing how neighboring vehicles move.

The study also highlights the difficulty of predicting sharp turns. Replacing the one-shot decoder with a recurrent GRU

reduced the error most on turning trajectories, but turns above 60° remain unsolved, although with only 87 such test samples this result is less conclusive. The GRU decoder, however, differs from the one-shot decoder in three ways at once, namely recurrence, displacement prediction and mode conditioning, so it remains unclear which of them causes the gain.

Future work could extend the edge features with relative velocity and heading. A larger change would be to replace the agent encoder with a transformer such as AgentFormer [2], which, instead of summarizing each agent's history in a single vector, treats every agent's position at every past time step as a separate input, so that the model can learn how agents influence each other over time.

REFERENCES

[1] T. Phan-Minh, E. C. Grigore, F. A. Boulton, O. Beijbom, and E. M. Wolff, "CoverNet: Multimodal behavior prediction using trajectory sets," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 14074–14083.

[2] Y. Yuan, X. Weng, Y. Ou, and K. Kitani, "AgentFormer: Agent-aware transformers for socio-temporal multi-agent forecasting," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021, pp. 9813–9823.

[3] M. Liang, B. Yang, R. Hu, Y. Chen, R. Liao, S. Feng, and R. Urtasun, "Learning lane graph representations for motion forecasting," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 541–556.

[4] H. Caesar *et al.*, "nuScenes: A multimodal dataset for autonomous driving," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11621–11631.

[5] A. Alahi, K. Goel, V. Ramanathan, A. Robicquet, L. Fei-Fei, and S. Savarese, "Social LSTM: Human trajectory prediction in crowded spaces," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 961–971.

[6] H. Cui, V. Radosavljevic, F.-C. Chou, T.-H. Lin, T. Nguyen, T.-K. Huang, J. Schneider, and N. Djuric, "Multimodal trajectory predictions for autonomous driving using deep convolutional networks," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2019, pp. 2090–2096.

[7] T. Salzmann, B. Ivanovic, P. Chakravarty, and M. Pavone, "Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 683–700.

[8] J. Gao, C. Sun, H. Zhao, Y. Shen, D. Anguelov, C. Li, and C. Schmid, "VectorNet: Encoding HD maps and agent dynamics from vectorized representation," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2020, pp. 11525–11533.

[9] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.

[10] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[11] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2014, pp. 1724–1734.